

VORWORT

partially yoonked from Jasmin Fässler, Nina Grässli & Jannis Tschan, motivated through Ian MacKaye

TODO:

– Minimalautomat

– Beispiele:

– Ist Grammatik eindeutig? (FS_2026)5))

– Parse-tree finden (FS_2022)5))

– Prüfungsaufgaben:

– HS_2021)4)

– FS_2022)4)

– Pumping Lemma diagrams (FS_2024)6))

VORGEHEN ZU AUFGABENTYPEN	
Frage	Antwort
Ist Sprache regulär?	Ja: DEA / NEA / RegEx Nein: Pumping Lemma
Ist Sprache kontextfrei?	Ja: Stackautomat / CNF / BNF Nein: Pumping Lemma (CFL)
Turing-Maschinen	Berechnungsgeschichte
Ist eine Sprache Turing-Vollständig?	Ja: TM-Simulation Nein: Halteproblem
Kann Problem von einem Computer gelöst werden?	Reduktion auf Halteproblem
Kann ein Programm entscheiden, ob zulässige Inputs eine Eigenschaft erfüllen?	Satz von Rice
Kann eine nichtdeterministische Maschine ... entscheiden?	NP-Verifizierer
Warum kann ein Problem nicht gelöst werden?	Reduktion auf NP-Vollständiges Problem

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

$A = (Q, \Sigma, \delta, q, F)$

– Zustände: $Q = \{q_0, q_1, \dots, q_n\}$

– Alphabet: Σ

– Übergangsfunktion: $\delta : Q \times \Sigma \rightarrow Q$

– Startzustand: $q \in Q$

– Akzeptierzustände: $F \subset Q$

0

1

Σ

q_0

q_2

q_1

q_1

q_0

q_2

q_2

q_1

q_0

δ

PUMPING LEMMA (DEA)

Um zu beweisen, dass eine Sprache **nicht** regulär ist

Vorgehen:

1) Annahme: L ist regulär

2) Gemäss Pumping Lemma gibt es die Pumping Length N

– Es darf keine Annahme über die konkrete Grösse von N gemacht werden!

3) Wähle ein Wort $w \in L$ mit $|w| \geq N$

– Die Definition **muss** N verwenden!

4) Aufteilung des Wortes gemäss Pumping Lemma

– $w = xyz$, $|xy| \leq N$, $|y| > 0$

5) Auswirkung des Pumpens

– $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)

6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 1: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

1) Annahme: L ist regulär

2) $\exists N \in \mathbb{N}$, Pumping Length

3) $w = 0^N 1^N$

4) Unterteilung $w = xyz$

5) Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 1 bleibt

6) $xy^kz \notin L$ für $k \neq 1$, Widerspruch zum Pumping Lemma

Beispiel 2: $L =$ alle korrekte Integraalausdrücke

1) Annahme: L ist regulär

2) $\exists N \in \mathbb{N}$, Pumping Length

3) $w = \int_{a_1}^{b_1} \int_{a_2}^{b_2} \dots \int_{a_N}^{b_N} dx_N \dots dx_2 dx_1 \in L$

4) Unterteile $w = xyz$, wobei $|y| \geq 1 \wedge |xy| \leq N \wedge xy^kz \in L$

5) Pumpen: Der Teil xy enthält keine dx_k , beim pumpen nimmt also nur die Anzahl der Integralzeichen zu, so dass kein korrekter Integraalausdruck stehen bleibt.

6) Der Widerspruch zeigt, dass die Annahme, L sei regulär, nicht haltbar ist. Also ist L nicht regulär.

6 Schritte des Pumping-Lemma-Beweises: Annahme (A) 1 Punkt, Pumping Length (N) 1 Punkt, Wort (W) 1 Punkt, Unterteilung (U) 1 Punkt, Widerspruch beim Pumpen (P) 1 Punkt, Folgerung (F) 1 Punkt.

NICHTDETERM. ENDLICHE AUTOMATEN (NEA)

$A = (Q, \Sigma, \delta, q, F)$

– Endliche Menge von Zuständen: Q

– Alphabet: Σ

– Übergangsfunktion: $\delta : Q \times \Sigma \rightarrow P(Q)$

– Startzustand: $q \in Q$

– Akzeptierzustände: $F \subset Q$

TODO:

Thompson-NEA

VERALLGEMEINERTER NEA (VNEA)

Begriff	Definition
Regulär	Es gibt einen DEA A , der L akzeptiert, also $L(A) = L$
Regulärer Ausdruck	Zeichenkette r zur Beschreibung einer regulären Sprache $L = L(r)$

Begriff	Definition
Reguläre Operationen	$L(r_1) \cup L(r_2) = L(r_1 \mid r_2)$ $L(r_1)L(r_2) = L(r_1r_2)$ $L(r_1)^* = L(r_1^*)$
Verallgemeinerter NEA	NEA, dessen Übergänge mit regulären Ausdrücken beschriftet sind
PRIMITIVE REGULÄRE AUSDRÜCKE	
Reguläre Ausdrücke für Wörter mit Länge ≤ 1	
$L = L(r)$	r
$\emptyset$	$\emptyset$
$\{\epsilon\}$	$\{\epsilon\}$
$\{a\}$	a
$\{o, s, t\}$	$[ost]$
$\{a, b, \dots, s\}$	$[a-s]$
Σ	Σ

KONTEXTFREIE SPRACHEN (CFL)

Kontextfrei: L kann nur von einem ND PDA erkannt werden

$G = (V, \Sigma, R, S)$

– V : Variablen

– Σ : Terminalsymbole (Alphabet)

– R : Regeln der Form $A \rightarrow x_1x_2\dots x_n$ mit $A \in V, x_i \in V \cup \Sigma$

– S : Startvariable

Parse Tree

$L = \{w \in \{0, 1\}^* \mid |w|_0 = |w|_1\}$

mit der Grammatik

$S \rightarrow 0S1 \mid 1S0 \mid SS \mid \epsilon$

Beispiel 4: Ist L kontextfrei?

$L = \{wxw^t \mid w, u \in \Sigma^* \wedge |w| \leq 3\}, \Sigma = \{a, b\}$

(w^t ist das gespiegelte Wort von w)

Die Sprache ist kontextfrei, weil die folgende kontextfreie Grammatik die Sprache erzeugen kann:

$S \rightarrow aSa \mid bSb \mid U$
 $U \rightarrow \epsilon \mid A \mid AA \mid AAA$
 $A \rightarrow a \mid b$

Alternativ kann man auch einen Stackautomaten angeben, der die Sprache akzeptiert.

Beispiel 5: $L = \{w \in \{0, 1\}^* \mid w = 0^k 10^k 10^k\}$

1) Annahme: L ist kontextfrei

2) Nach dem Pumping Lemma $\exists N \in \mathbb{N}$, Pumping Length

3) Wähle das Wort $w = 0^N 10^N 10^N 10^N$

4) Nach dem Pumping Lemma gibt es eine Aufteilung von w in fünf Teile $w = uvxyz$ derart, dass $|vxy| \leq N \wedge |vy| \geq 1$. Ausserdem ist jedes gepumpte Wort $uv^kxy^kz \in L$.

5) Da sich die Anzahl der Einsen beim Pumpen nicht ändern darf, müssen v und y vollständig in einem Nullen-Block enthalten sein. Daher kann sich nur die Anzahl der Nullen in höchstens zwei Nullen-Blöcken ändern. Die beiden Blöcke müssen wegen $|vxy| \leq N$ ausserdem benachbart sein.

Zum ersten Nullen-Block gehört der dritte, der gleich viele Nullen enthalten muss, zum zweiten gehört der vierte. Wie auch immer die beiden Blöcke gewählt werden, ändert sich die Anzahl Nullen in den Blöcken, aber nicht in den zugehörigen Blöcken. Das gepumpte Wort kann also nicht mehr in L sein.

6) Dieser Widerspruch zeigt, dass die Annahme, L sei kontextfrei, nicht haltbar ist. Also ist L nicht kontextfrei.

Beispiel 6:

TODO:

FS_2024)6)

CHOMSKY-NORMALFORM (CNF)

– S kommt auf der rechten Seite nicht vor

– Jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$

– $S \rightarrow \epsilon$ ist erlaubt

UMWANDLUNG

1) Neue Startvariable $S_0 \rightarrow S$ (wenn nötig)

2) ϵ -Regeln:

$A \rightarrow \epsilon$
 $B \rightarrow AC \} \Rightarrow \begin{cases} B \rightarrow AC \\ \rightarrow \Delta C \end{cases}$

3) Unit-Rules:

$A \rightarrow B$
 $B \rightarrow CD \} \Rightarrow \begin{cases} A \rightarrow CD \\ B \rightarrow CD \end{cases}$

4) Verkettungen:

$A \rightarrow u_1u_2\dots u_n$
 $A \rightarrow u_1A_1, A_1 \rightarrow u_2A_2, \dots, A_{n-2} \rightarrow u_{n-1}u_n$
falls u_i ein Terminalsymbol: $A_{i-1} \rightarrow U_iA_i, U_i \rightarrow u_i$

Beispiel 7: $S \rightarrow ASA \mid aB, A \rightarrow B \mid S, B \rightarrow b \mid \epsilon$

1) Startvariable

$S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB$
 $A \rightarrow B \mid S$
 $B \rightarrow b \mid \epsilon$

2) ϵ -Regel

$S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB \mid a$
 $A \rightarrow B \mid \epsilon \mid S$
 $B \rightarrow b$

3) Unit-Regel

$S_0 \rightarrow S$
 $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $A \rightarrow B \mid S$
 $B \rightarrow b$

$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a$
 $B \rightarrow b$

$\rightsquigarrow$

$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $A \rightarrow b \mid ASA \mid SA \mid AS \mid aB \mid a$
 $B \rightarrow b$

4) Komplexe Regeln

$S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a$
 $AA_1 \rightarrow SA$
 $U \rightarrow a$
 $B \rightarrow b$

ABLEITUNGSDREIECK

Ideen

– Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$

– Variablen in Tabelle füllen

Prinzip

– Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)

– Das Feld wird mit den Variablen gefüllt, aus denen das Teilwort abgeleitet werden kann.

Beispiel

Parsen des Wortes ()(())

$S \rightarrow AB \mid CD \mid CU \mid SS$

$U \rightarrow SD$

$A \rightarrow ($

$B \rightarrow)$

$C \rightarrow [$

$D \rightarrow]$

Definitionen

Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heissen **korrespondierende Felder** im Ableitungsdreieck des Feldes (k, l) .

BNF

– Variablen: **<variablen-name>**

– Einzelne Zeichen: A

– Zeichenketten: 'BEISPIEL'

– Regeln: **<variablen-name> ::= Ausdruck**

– Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch $|$

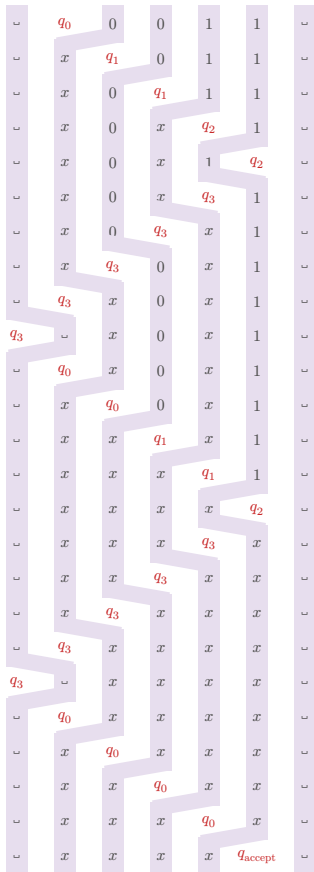
STACKAUTOMAT (PDA)

$P = (Q, \Sigma, \Gamma, \delta, q_0, F)$

AutoSpr | FS26

Seite 1

- $$L = \{0^n 1^n \mid n \geq 0\}, w = 0011$$



Turing-entscheidbare Sprache: L heisst **Turing-entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$.

Eine Eigenschaft P Turing-erkennbarer Sprachen heisst **nichttrivial**, wenn es zwei Turing-Maschinen M_1 und M_2 gibt, wobei M_1 die Eigenschaft hat und M_2 nicht.

Die Eigenschaft einer Sprache, links-kürzbar zu sein, ist eine nichttriviale Eigenschaft. Die Liste der Wörter in der Aufgabenstellung bildet eine links-kürzbare Sprache, die Sprache bestehend nur aus dem Wort BIBER hat diese Eigenschaft nicht. Der Satz von Rice besagt jetzt, dass man kein Programm schreiben kann, welches entscheiden

Für einen polynomiellen Verifizierer brauchen wir ein Lösungszertifikat, wir nehmen dafür die Zahlen, die in den Feldern stehen. Folgende Verifikationsschritte sind noch nötig:

	Schritt	Aufwand
1	Sudoku-Verifizierer	polynomiell
2	Für jedes Feld überprüfen, ob die Summe der Zahlen im gleichen Käfig eine Quadratzahl ist.	$O(n^4 \cdot n^4)$
	Total	polynomiell

Entscheidbarkeit (E) 1 Punkt, Verifizierer (V) 1 Punkt, Zertifikat (Z) 1 Punkt, bestehende Sudoku Regeln sind in polynomieller Zeit verifizierbar (S) 1 Punkt, Verifikation von zwei zusätzlichen Regel (R) 1 Punkt, Laufzeit polynomiell (L) 1 Punkt.

m-SUDOKU

Entscheidbar? Ja: $n = m^4 =$ Anzahl Felder

Zertifikat: Vollständig ausgefülltes Sudoku

Vorgehen:

	Schritt	Aufwand
1	Jedes Feld: Zeichen kommt auf der Zeile nicht mehr vor	$O(n^4 \cdot n^2)$
2	Jedes Feld: Zeichen kommt in der Spalte nicht mehr vor	$O(n^4 \cdot n^2)$
3	Jedes Feld: Zeichen kommt im Unterquadrat nicht vor	$O(n^4 \cdot n^2)$
	Total	$O(n^6)$

NURIKABE

Entscheidbar? Ja: $n = uv =$ Anzahl Felder. Alle Belegungen des Spielfeldes mit schwarzen Feldern können überprüft werden, ob sie die Regeln einhalten.

Zertifikat: Liste der schwarzen Felder

Vorgehen:

	Schritt	Aufwand
1	Für jedes Zahlfeld ($O(n)$) verwendet man einen Markieralgorithmus, der alle zum Gebiet dieses Zahlfeldes gehörenden weissen Felder bestimmt ($O(n)$ Durchgänge mit Aufwand $O(n)$).	$O(n^3)$
2	Trifft dieser Algorithmus auf ein weiteres Zahlfeld, ist der erste Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$O(n)$
3	Weicht die Zahl der gefundenen weissen Felder vom Inhalt des Zahlfeldes ab, ist der zweite Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$n \cdot O(n)$
4	Ebenfalls mit einem Markieralgorithmus wird überprüft, ob das schwarze Gebiet zusammenhängend ist.	$O(n^2)$
5	Für jedes schwarze Feld wird überprüft, ob es Teil eines 2×2 -Tümpels ist.	$O(n)$
	Total	$O(n^3)$

DAMEN

TODO:

REDUKTION

Polynomielle Reduktion: $A \leq_P B$. Lies: A ist polynomiell leichter entscheidbar als B .

TODO:

NP-VOLLSTÄNDIG

Reduktion auf NP-Vollständiges Problem

Zu beachten (Punkteverteilung):

- Lösungsprinzip der Reduktion erwähnen (1P)
- Auswahl eines geeigneten Vergleichsproblems (1P)
- Reduktionsabbildung (*P)
- Schlussfolgerung: NP-Vollständig und somit nicht effizient lösbar (1P)

TODO:

TURING-VOLLSTÄNDIG

Eine Sprache A heisst eine **Programmiersprache**, wenn es eine Abbildung $c : A \mapsto \Sigma^*$ gibt, wobei $c(w)$ ein Programm für eine universelle Turing-Maschine ist.

Eine Programmiersprache heisst **Turing-vollständig**, wenn in ihr jede beliebige Turing-Maschine simuliert werden kann.

KARP-KATALOG

PROBLEME MIT k ZAHLEN

VERTEX-COLORING

EXACT-CLIQUE-COVER

VERTEX-COVER

SET-COVERING

STEINER-TREE

FEEDBACK-NODE-SET

FEEDBACK-ARC-SET

GRAPH-ÜBERDECKUNGEN

VERTEX-COLORING

CLIQUE-COVER

EXACT-CLIQUE-COVER

K-CLIQUE

VERTEX-COVER

Handeln von einer Familie $(S_i)_{i \in I}$ von nichtleeren endlichen Mengen. In diesen Problemen geht es um die Existenz einer Teilfamilie mit bestimmten Eigenschaften.

SET-PACKING

Wie viele der Mengen S_i kann man in U hineinpacken, ohne dass sie sich überlappen? Gibt es k Teilmengen in S , welche disjunkt sind und die Menge U abdecken? **Beispiel:** Medizinische Studie: 17 Allergene wählen, sodass jeder Proband maximal auf eines davon reagiert.

Lösung: $I =$ Allergene, $S_i =$ auf Allergien i allergische Probanden, $J =$ ausgewählte Allergene, $S_i \cap S_j = \emptyset \rightarrow$ Ausschlussbedingung zwischen Allergenen

SET-COVERING

Gibt es eine Unterfamilie bestehend aus k Mengen, die die gleiche Vereinigung U hat? Kann man k Teilmengen bilden, welche die Menge S komplett abdecken? **Beispiel:** Ein Tourist möchte mit dem Besuch von nur k Aussichtspunkten alles sehen, was man an Sehenswürdigkeiten von allen Aussichtspunkten aus sehen könnte.

Lösung: $U =$ Alle Sehenswürdigkeiten, $i \in I =$ Aussichtspunkt i , $S_i =$ sichtbare Sehenswürdigkeiten von i aus, $J \subset I =$ ausgewählte Aussichtspunkte, $\bigcup_{j \in J} S_j = U =$ Bedingung, alles sehen zu können

EXACT-COVER

Gibt es eine Unterfamilie von Mengen, die disjunkt sind, aber die gleiche Vereinigung haben? Jedes Element in U soll genau in einer der Teilmengen der Familie S vorkommen. Die gesuchte Menge bildet eine exakte Überdeckung.

Beispiel: Es stehen binäre Eigenschaften von Kunden zur Verfügung, z.B. ob sie ein bestimmtes Produkt gekauft haben oder volljährig sind. Ziel: Eine Teilmenge von Kriterien finden, dass jeder Kunde genau eine der Eigenschaften hat.

Lösung: $I =$ Eigenschaften, $(S_i)_{i \in I} =$ Kunden, auf die die Eigenschaft zutrifft, $J \subset I =$ Teilmenge

HITTING-SET

Sucht zu U eine Menge von Punkten (Elemente), die jede Menge der Familie in genau einem Punkt treffen. Gibt es eine Menge $W \subset U$ derart, dass W mit jeder Menge S_i nur ein Element gemeinsam hat?

Beispiel: Finden eines einzelnen Vertreters für Gruppen von Menschen.

Lösung: $i \in I =$ Gruppierungskriterien, $S_i =$ Menschen in Gruppe i , $W \subset \bigcup_{i \in I} S_i =$ Teilmenge von Vertretern, $|W \cap S_i| = 1 \forall i \rightarrow H$ hat mit jeder der Mengen S_i genau einen Vertreter gemeinsam

VERTEX-COVER

Gibt es eine Menge W von k Knoten derart, dass jede Kante von G einen Endpunkt in W hat?

Beispiel: Kontrolle des Verkehrsnetzes: Mitarbeiter haben ihre Basis an einzelnen Knotenpunkten. Ziel: An welchen Stationen muss man Kontrolleure stationieren, damit jede Strecke bei einem Kontrolleur endet?

Lösung: Knoten $V =$ Stationen, Kanten $E =$ Strecke, $k =$ Anzahl Kontrolleure, $W \subset V =$ Knoten mit Kontrolleur

AUFTEILUNG IN ZWEI MENGEN

PARTITION

Gibt es eine Unterteilung einer Liste S von ganzen Zahlen in zwei disjunkte Listen mit gleicher Summe?

Beispiel: Unterteilung der Prominenten + Entourage auf zwei Säle des Film-Festivals.

Lösung: $i \in I =$ Promi, $c_i =$ Anz. Mitglieder seiner Entourage, $A =$ Stars samt Entourage für Saal A, $B =$ für Saal B, $I = A \cup B, \sum_{i \in A} c_i = \sum_{i \in B} c_i$

MAX-CUT

Gegeben ein Graph $G = (V, E)$ mit einer Gewichtsfunktion $\varphi: E \rightarrow \mathbb{Z}$ der Kanten und einer ganzen Zahl $k \in \mathbb{Z}$, gibt es eine Aufteilung der Knotenmenge $V = V_1 \cup V_2$ in zwei disjunkte Teilmengen, so dass das Gewicht der Kanten, die V_1 und V_2 verbinden, mindestens w ist:

$$\varphi(V_1, V_2) = \sum_{v_1 \in V_1, v_2 \in V_2, e = \{v_1, v_2\} \in E} \varphi(e) \geq k?$$

Beispiel: Feindliche Übernahme einer Firma, mit resultierender Aufteilung der Abteilung, sodass diese möglichst ineffizient miteinander kommunizieren können.

Lösung: $V =$ Abteilung, $E =$ Kommunikationsbeziehung, $\varphi =$ Kommunikationsvolumen

KOMBINATORISCHE OPTIMIERUNG

Handelt um das Finden eines Maximums oder Minimums einer Zielfunktion.

STEINER-TREE

Gibt es zu einem Graphen $G = (V, E)$ mit einer Gewichtsfunktion $\varphi: E \rightarrow \mathbb{Z}$, einer Teilmenge $R \subset V$ der Knoten und einer Zahl k einen Teilbaum $T = (V_1, E_1) \subset G$ des Graphen, der alle Knoten von R enthält und dessen Kantengewicht $\sum_{e \in E_1} \varphi(e) \leq k$ ist?

Beispiel: Stromnetzausbau zwischen Ortschaften gegeben eines Budgets.

Lösung: STEINER-TREE = Stromnetz, Knoten $V =$ Ortschaften, Knoten $R \subset V =$ zu erschliessende Ortschaften, Gewicht $\varphi =$ Baukosten einer Verbindung, Max. Gewicht $k =$ Budget

SEQUENCING

Die Komponenten der drei Vektoren

$$\text{Laufzeiten: } T = (T_1, \dots, T_p) \in \mathbb{Z}^p$$
$$\text{Deadlines: } D = (D_1, \dots, D_p) \in \mathbb{Z}^p$$
$$\text{Strafen: } P = (P_1, \dots, P_p) \in \mathbb{Z}^p$$

beschreiben je einen Job. Job i hat Laufzeit T_i , sollte zur Zeit D_i abgeschlossen sein und kostet die Strafe P_i , wenn er nicht rechtzeitig fertig ist. Die Jobs werden sequenziell ohne Unterbruch abgearbeitet. Ist es möglich, die Reihenfolge der Ausführung der Jobs so zu planen, dass die entstehenden Strafen $\leq k$ sind?

Beispiel: Eine Firma hat eine bestimmte Anzahl laufende Verträge. Es ist nicht möglich, alle Verträge in einer bestimmten Zeit abzuarbeiten. Sie versucht also möglichst viele Verträge in der verbleibenden Zeit abzuarbeiten, die eine hohe Entlohnung haben.

Lösung: $T =$ Zeitaufwände für Vertragserfüllung, $P =$ Entlohnung der Verträge, $D =$ Deadlines der Verträge

PFAD IN GRAPHEN

Ein hamiltonscher Pfad in einem Graphen $G = (V, E)$ ist ein Pfad, der jeden Knoten des Graphen genau einmal besucht. Ein solcher Pfad definiert eine Reihenfolge $a_1, \dots, a_n$ von Knoten, so dass jede Kante von a_i nach a_{i+1} in E ist. Für einen gerichteten Graphen sind dies die Kanten $(a_i, a_{i+1}) \in E$, für einen ungerichteten Graphen gilt $\{a_i, a_{i+1}\} \in E$. Ein hamiltonscher Zyklus ist ein geschlossener hamiltonscher Pfad.

HAMPATH

Gibt es einen gerichteten hamiltonschen Pfad im gerichteten Graphen G ?

Beispiel: Ganze Schweiz bereisen, ohne eine Stadt mehrmals zu besuchen

Lösung: $G =$ Liniennetz, $V =$ Stadt, $E =$ ÖV-Verbindung, Hamiltonscher Pfad = Reise

HAMCIRCUIT

Wie HAMPATH, aber zusätzlich muss der Pfad geschlossen sein

UHAMPATH

Wie HAMPATH, aber ungerichtet

UHAMCIRCUIT

Wie UHAMPATH, aber zusätzlich muss der Pfad geschlossen sein

GERICHTETE GRAPHEN

Es handelt sich um die Eigenschaften von Zyklen in gerichteten Graphen $G = (V, E)$. Die Kantenmenge E besteht aus gerichteten Kanten (a, e) mit $a, e \in V$. Ein gerichteter Zyklus in G ist eine Folge von Knoten $a_0, \dots, a_n = a_0$, die jeweils Enden von Kanten sind, also $(a_i, a_{i+1}) \in E, i = 0, \dots, n-1$

FEEDBACK-NODE-SET

Gibt es zu einem gerichteten Graphen G und einer Zahl k eine Menge $W \subset V$ von k Knoten derart, dass jeder geschlossene Zyklus durch einen dieser Knoten geht?

Beispiel: Es gibt mehrere Buslinien. Wo muss das Putzpersonal platziert werden, damit alle Linien geputzt werden können? Man möchte möglichst wenig Personal einsetzen.

Lösung: $V =$ Stationen, $W \subset V =$ Stationen, von denen aus alle Linien erreicht werden

FEEDBACK-ARC-SET

Gibt es zu einem gerichteten Graphen G und einer Zahl k eine Menge $F \subset E$ von k Kanten derart, dass jeder geschlossene Zyklus aus dieser Kanten enthält?

LOGIK

SAT

Gegeben einer aussagenlogischen Formel, gibt es eine Zusammensetzung der Variablenwerte, die die Aussage «Wahr» werden lassen?

Beispiel: Elektriker: Konfiguration der Schalter zweier Stromkreise, die in allen Räumen Licht brennen lassen sollten

Lösung: $x_i =$ Schalter, $T \vee F =$ Stromkreise (werte der Variable x_i , Lampen des ersten Kreises leuchten, wenn $x_i = T$), $x_i \vee \dots \vee x_{i_k} = \text{ob Licht im Raum der mit } x_i \text{ verbundenen Schalter brennt}$

3SAT

Wie SAT, nur hat jede Klausel der Normalform maximal 3 Literale.

Beispiel: SAT $\rightarrow$ 3SAT

Lösung:

$$(x_1 \vee x_2 \vee x_3 \vee x_4) \wedge (x_5 \vee x_6)$$
$$\rightarrow (x_1 \vee x_2 \vee z_1) \wedge (\bar{z}_1 \vee x_3 \vee x_4) \wedge (x_5 \vee x_6 \vee x_6)$$

WEITERE

SUBSET-SUM

Gegeben einer Liste S von natürlichen Zahlen $s_i \in \mathbb{N}$ und einer natürlichen Zahl $t \in \mathbb{N}$, ist es möglich eine Teilmenge $T \subset S$ auszuwählen, sodass die Elemente von T die Summe $t = \sum_{s \in T} s$ haben?

Beispiel: Rucksack-Problem (Ziel: Rucksack füllen)

Lösung: $S =$ Grössen der Gegenstände, $t =$ Max. Platz im Rucksack, $T \subset S =$ Grössen der Gegenstände, die in den Rucksack passen

3D-MATCHING

Gegeben sind drei endliche Mengen X, Y und Z mit gleich vielen $n = |X| = |Y| = |Z|$ Elementen und $T \subset X \times Y \times Z$. Gibt es eine n -elementige Teilmenge $M \subset T$ derart, dass keine zwei Tripel von M in einer Koordinate übereinstimmen?

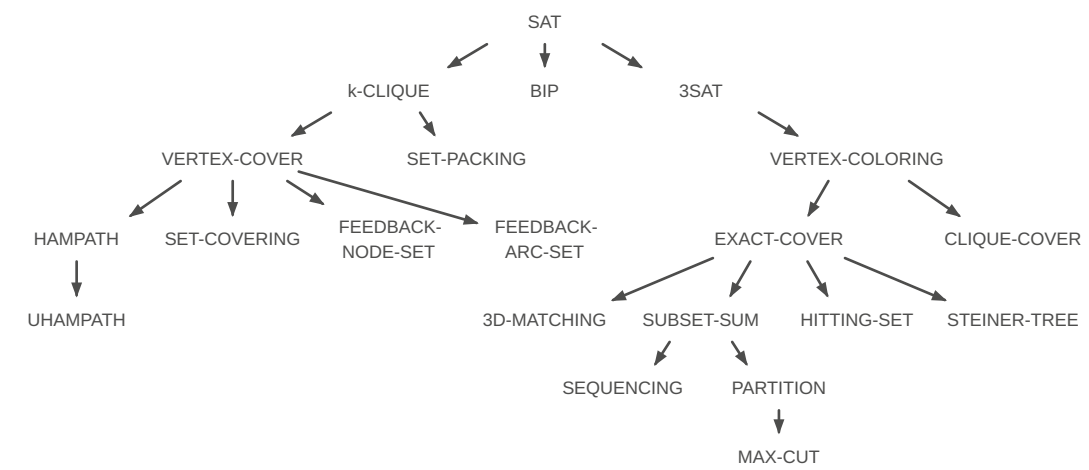
Beispiel: Ein Koch hat n Rezepte für Vorspeisen, Hauptspeisen und Desserts. Nicht alle Vorspeisen lassen sich mit jeder Hauptspeise kombinieren, dasselbe gilt auch für Desserts. Er möchte n Menus zusammenstellen, sodass jedes Rezept in genau einem der Menus vorkommt.

Lösung: $X =$ Vorspeisen, $Y =$ Hauptspeisen, $Z =$ Desserts, $T \subset X \times Y \times Z =$ Menge aller Menus, $M \subset T =$ Menge der zusammengestellter Menus

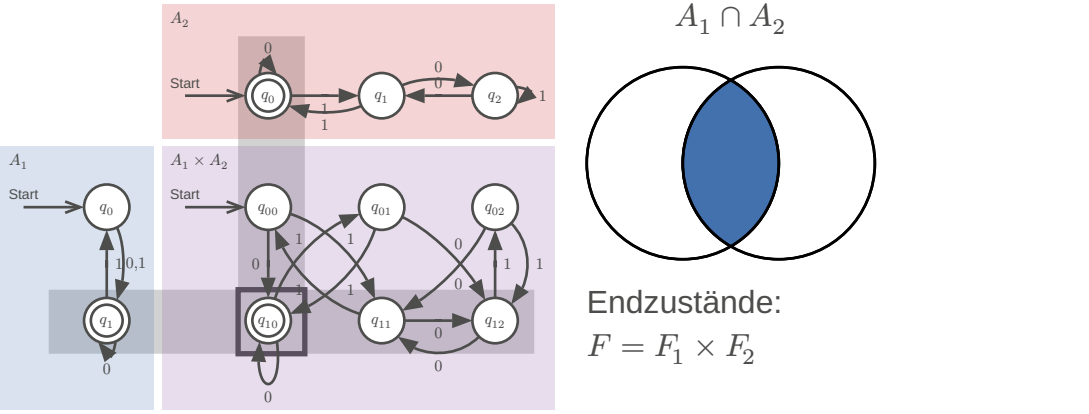
BIP

BIP = Binary Integer Programming

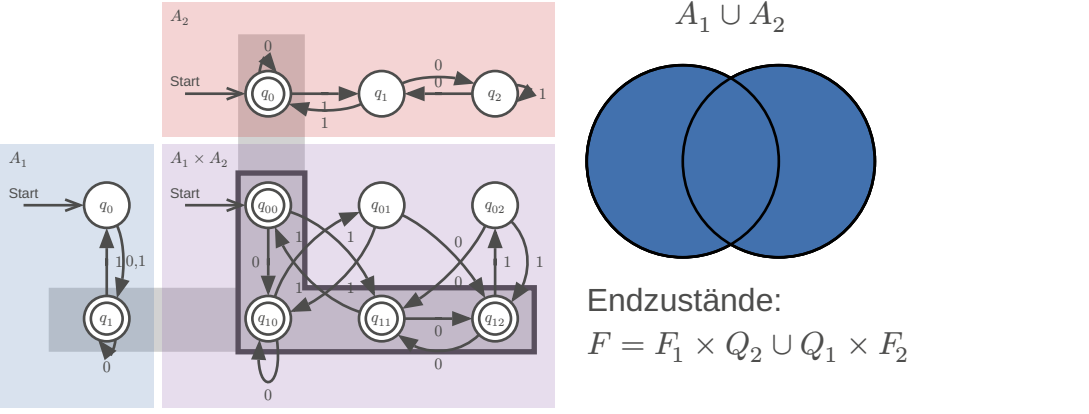
Zu einer ganzzahligen Matrix C und einem ganzzahligen Vektor d , ist ein binärer Vektor x zu finden mit $C \cdot x = d$



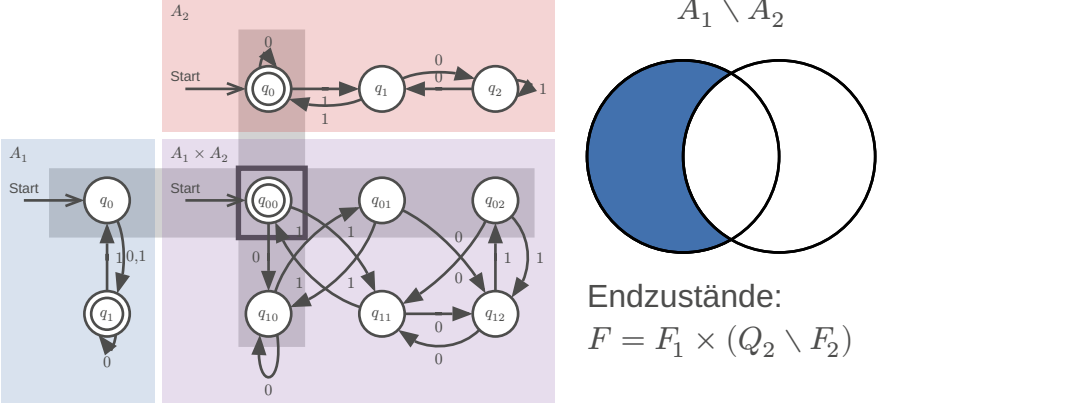
MENGEN
SCHNITTMENGE $L(A_1) \cap L(A_2)$



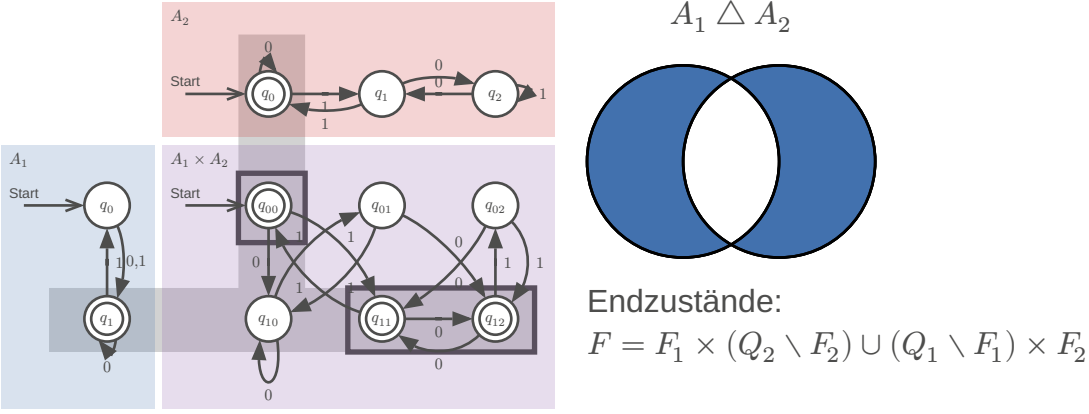
VEREINIGUNGSMENGE $L(A_1) \cup L(A_2)$



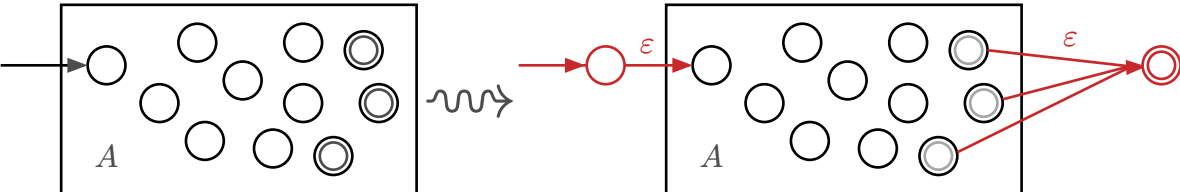
DIFFERENZMENGE $L(A_1) \setminus L(A_2)$



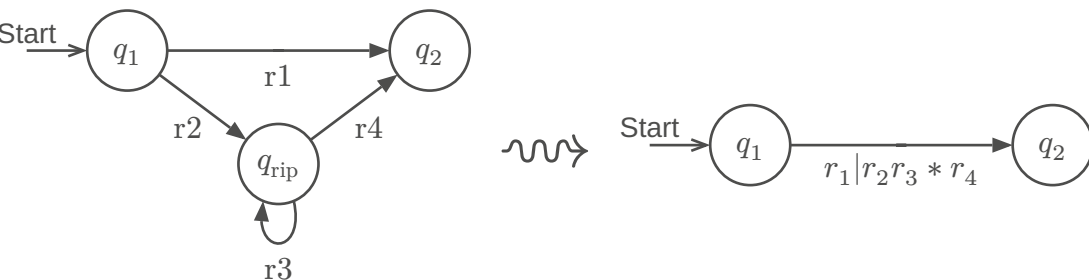
SYMMETRISCHE DIFFERENZ $L(A_1) \triangle L(A_2)$



Keine Übergänge nach q_0 und nur ein Akzeptierzustand

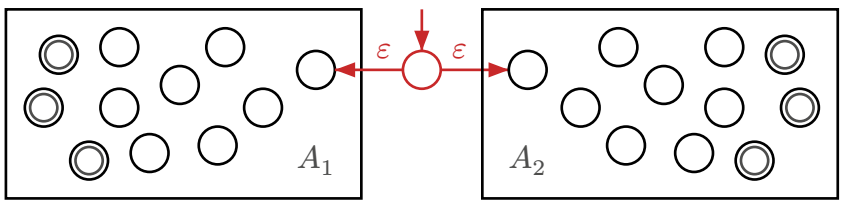


Nach Entfernen aller Zwischenzustände $q_{rip} \in Q$ bleibt ein regulärer Ausdruck r von A



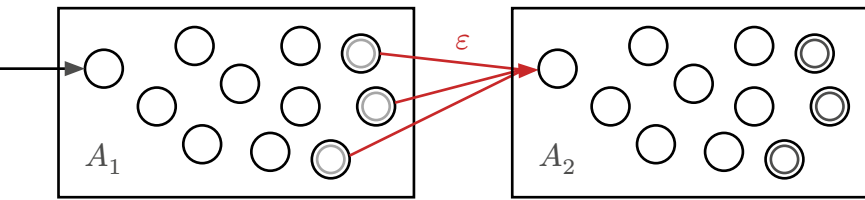
ALTERNATIVE

$$\begin{aligned} L &= L_1 \cup L_2 \\ &= L(A_1) \cup L(A_2) \\ &= L(A_1) \mid L(A_2) \end{aligned}$$



VERKETTUNG

$$\begin{aligned} L &= L_1 L_2 \\ &= L(A_1) L(A_2) \\ &= \{w_1 w_2 \mid w_i \in L_i\} \end{aligned}$$



$$\begin{aligned} L^* &= \{\epsilon\} \cup L \cup L^2 \cup \dots \\ &= \bigcup_{k=0}^{\infty} L^k \end{aligned}$$

